Thin proxy — step-by-step setup (no local installs)

Everything here runs in **Azure Cloud Shell** (a browser terminal with `az` and `func` preinstalled). Nothing is installed on your machine.

Before you start

- An Azure subscription where you can create resources (**Contributor** rights). If IT owns Azure, you may need them to grant you a resource group.
- Your Cat credentials (from `.env`): client ID, client secret, and scope.
- The `azure-function/` folder committed and pushed to GitHub (it contains **no secrets**, so it's safe in the public repo).

Step 1 — Push the function code to GitHub

From your machine, commit and push so Cloud Shell can pull it:

```
git add azure-function0
git commit -m "Add read-only proxy Azure Function"
git push
```

(No secrets are in these files — credentials come from Key Vault.)

Step 2 — Open Azure Cloud Shell

Go to **portal.azure.com** → click the `>_` icon (top bar) → choose **Bash**. First run creates a small storage account for the shell — accept.

Step 3 — Pick names and create the resources

Paste into Cloud Shell. Names for storage/Key Vault/function must be globally unique — add digits if one is taken.

```
RG=rg-cat-broker
LOC=eastus
STORAGE=catbrokerstg01          # lowercase, <=24 chars, globally unique
KV=cat-broker-kv01              # globally unique
FUNC=cat-broker-func01          # globally unique -> becomes your URL host

az group create -n $RG -l $LOC

az storage account create -n $STORAGE -g $RG -l $LOC --sku Standard_LRS

az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization false

az functionapp create -n $FUNC -g $RG \
  --storage-account $STORAGE \
  --consumption-plan-location $LOC \
  --runtime python --runtime-version 3.11 \
  --functions-version 4 --os-type Linux
```

Step 4 — Give the function access to the secret

```
az functionapp identity assign -n $FUNC -g $RG

PRINCIPAL=$(az functionapp identity show -n $FUNC -g $RG --query principalId -o tsv)
az keyvault set-policy -n $KV --object-id $PRINCIPAL --secret-permissions get list
```

Step 5 — Store the Cat credentials in Key Vault

Replace the placeholders with your real values.

```
az keyvault secret set --vault-name $KV --name CatClientId     --value "PASTE-CLIENT-ID"
az keyvault secret set --vault-name $KV --name CatClientSecret --value "PASTE-CLIENT-SECRET"
```

Step 6 — Configure the app settings

Secrets are Key Vault references; the rest are plain values. Replace `PASTE-CLIENT-ID` in the scope.

```
az functionapp config appsettings set -n $FUNC -g $RG --settings \
  "CAT_CLIENT_ID=@Microsoft.KeyVault(SecretUri=https://$KV.vault.azure.net/secrets/CatClientId/)" \
  "CAT_CLIENT_SECRET=@Microsoft.KeyVault(SecretUri=https://$KV.vault.azure.net/secrets/CatClientSecr
  "CAT_SCOPE=PASTE-CLIENT-ID/.default" \
  "CAT_TENANT_ID=ceb177bf-013b-49ab-8a9c-4abce32afc1e" \
  "CAT_DEFAULT_PARTY_NUMBER=ZZIO"
```

Step 7 — Deploy the code

```
git clone https://github.com/leakydata/asset-management-v2
cd asset-management-v2/azure-function
func azure functionapp publish $FUNC
```

(If a dependency build error appears, re-run with `--build remote`.)

Step 8 — Test it

```
KEY=$(az functionapp keys list -n $FUNC -g $RG --query "functionKeys.default" -o tsv)
curl "https://$FUNC.azurewebsites.net/api/search?serial=9303&code=$KEY"
```

You should get the same JSON the Cat API returns. **The proxy is live.**

Step 9 — (Recommended) Turn on per-user sign-in

In the Portal: **Function App → Settings → Authentication → Add identity provider → Microsoft → your tenant**. This replaces the shared function key with each user's Entra identity.

Step 10 — Point Excel at the proxy

In the workbook, set `CatSearch` to call `https://<FUNC>.azurewebsites.net/api/search` with the function key, and **delete the client secret from the Config sheet**. The lookup/batch logic stays the same.

Where the money goes

Consumption plan + this volume = effectively **$0-$5/month** (mostly the storage account). Don't add API Management unless you need an enterprise gateway.

What you just built

A service that holds the Cat secret in Key Vault, authenticates your users, and exposes **only** read-only search — so Excel can be distributed company-wide with no credentials inside it, and writes are impossible by construction.